RAPPORT DE TRAVAUX PRATIQUES — TP 2

Restructuration en

Monolithe Modulaire

Spring Boot · MapStruct · DTO · Architecture par Domaines

Réalisé par : Chaymae HICHAMI-ALAOU

Encadrant : Pr. BE ELBAGHAZAOUI

Module : Architecture Logicielle

TP 2 — Suite du TP 1 : Application Monolithique → Monolithe Modulaire

1. Introduction

Ce rapport présente la **restructuration de l'application e-commerce monolithique initiale** (TP 1) en une **architecture monolithique modulaire**. Cette approche intermédiaire entre le monolithe classique et les microservices permet de définir des **frontières claires entre les domaines métier** — Produits, Commandes, Clients — tout en conservant un déploiement unique.

La transition repose sur trois piliers techniques : la **modularisation par package métier**, l'introduction des **DTOs (Data Transfer Objects)** pour sécuriser l'API, et l'utilisation de **MapStruct** pour automatiser les conversions Entité ↔ DTO.

> **Contexte :** Cette architecture modulaire est une étape de maturation avant une éventuelle migration vers des microservices. Elle réduit le couplage sans la complexité opérationnelle d'un système distribué.

2. Objectifs Techniques

Quatre objectifs techniques structurent cette refactorisation :

Objectif	Technique	Bénéfice attendu
Modularisation	`Packages par domaine`	Séparation claire : Product / Order / Customer. Chaque domaine est autonome et isolé.
Abstraction des données	`DTO (Data Transfer Objects)`	L'API n'expose jamais les entités JPA directement. Protection du modèle interne.
Mapping automatique	`MapStruct`	Conversion Entité ↔ DTO générée à la compilation. Zéro boilerplate, performances natives.
Communication inter-modules	`Interfaces de services publics`	Le module Order appelle le service public de Product, jamais son Repository. Couplage minimal.

3. Architecture et Structure du Projet

Le projet est organisé sous le package racine `com.ecommerce.monolith`, décomposé en sous-packages par domaine métier. Chaque domaine (`product`, `order`, `customer`) contient ses propres couches internes, garantissant une totale indépendance fonctionnelle.

A. Couches par Module

Chaque domaine métier suit la même structure en six couches :

Couche	Composant	Annotation	Rôle
Model	`Product.java`	`@Entity`	Entité JPA ↔ table PostgreSQL
DTO	`ProductDTO.java`	—	Objet exposé par l'API REST
Mapper	`ProductMapper.java`	`@Mapper (MapStruct)`	Conversion automatique Entity ↔ DTO
Repository	`ProductRepository.java`	`@Repository`	Accès aux données (Spring Data JPA)
Service	`ProductServiceImpl.java`	`@Service` `@Transactional`	Logique métier + orchestration
Controller	`ProductController.java`	`@RestController`	Exposition des endpoints REST

Règle d'or : Un module ne doit jamais importer directement le Repository d'un autre module. Toute communication inter-domaines passe obligatoirement par l'**interface de service publique**.

4. Implémentation Détaillée

A. MapStruct — Interface de Mapping Automatique

MapStruct est une bibliothèque de génération de code qui crée automatiquement, à la **compilation**, le code de conversion entre entités et DTOs. Contrairement à des frameworks comme ModelMapper (réflexion à l'exécution), MapStruct génère du bytecode natif — sans overhead de performance.

Comparaison des approches de mapping :

Critère	Mapping manuel	ModelMapper	MapStruct
Code à écrire	getters/setters manuels	Aucun	**Interface seulement**
Performance	Optimale	Faible (réflexion)	**Optimale (compilé)**
Détection d'erreurs	Runtime	Runtime	**Compilation**
Compatibilité Lombok	Manuelle	Partielle	**Totale (avec config)**
Recommandation	Petits projets	Prototypes	**Production**

B. Couche Service Refactorisée

La couche service suit désormais le **pattern Interface + Implémentation**. L'interface `ProductService` constitue le **contrat public du module**, accessible par les autres modules. L'implémentation `ProductServiceImpl` reste interne au module et inconnue des appelants extérieurs.

C. Gestion des Commandes — Module Order

Le module `order` illustre la **communication inter-modules**. Lors de la création d'une commande, le service `OrderServiceImpl` doit vérifier la disponibilité du stock via le module `product`. Cette communication se fait exclusivement par l'**interface publique** — jamais par accès direct au Repository du module Product.

> **Règle d'architecture respectée :** Le module `Order` dépend uniquement de l'**interface ProductService**, jamais de `ProductRepository` ni de l'entité `Product`. Si le module Product change son modèle interne, le module Order n'est pas affecté.

D. Contrôleur REST — ProductController

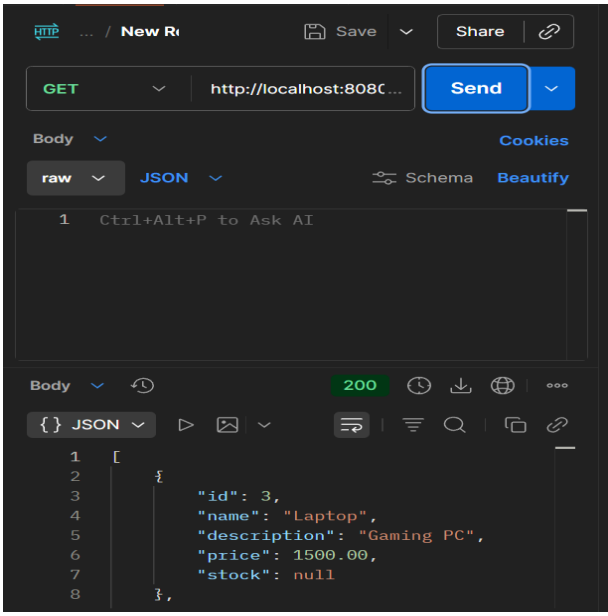
Le contrôleur ne manipule que des **DTOs** — ni entités JPA, ni logique métier. Son rôle se limite à la réception des requêtes HTTP, la délégation au service, et le formatage des réponses .

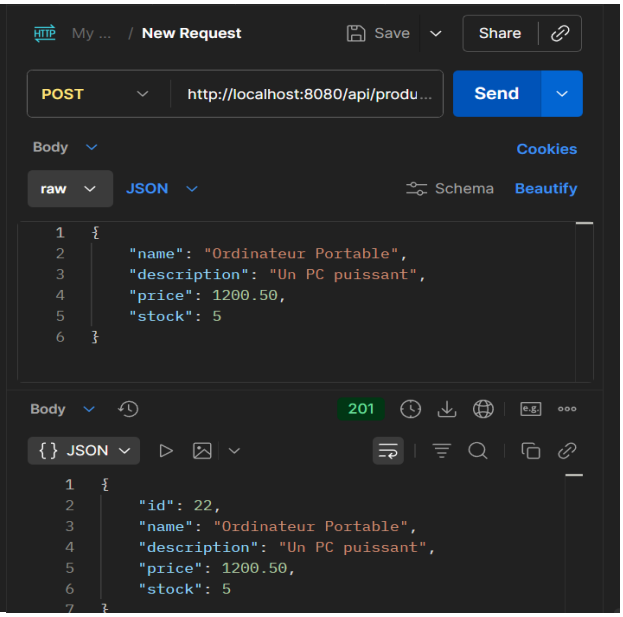
> **Bonne pratique :** Le `@ControllerAdvice` est un composant transversal (cross-cutting concern). Il doit rester dans un package partagé (`common/exception`) pour ne pas appartenir à un domaine métier spécifique.

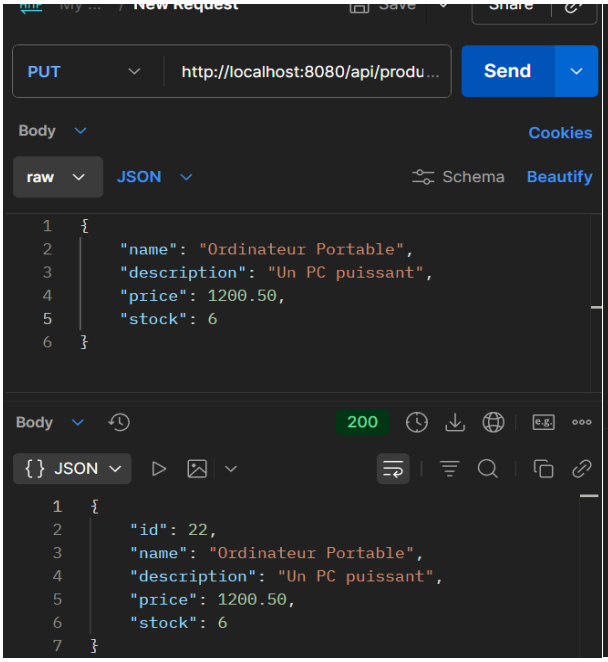
6. Tests et Validation

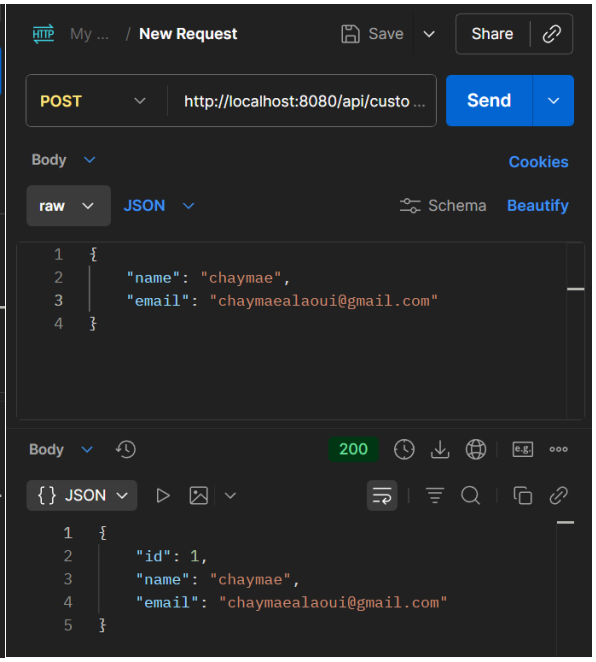
A. Endpoints disponibles

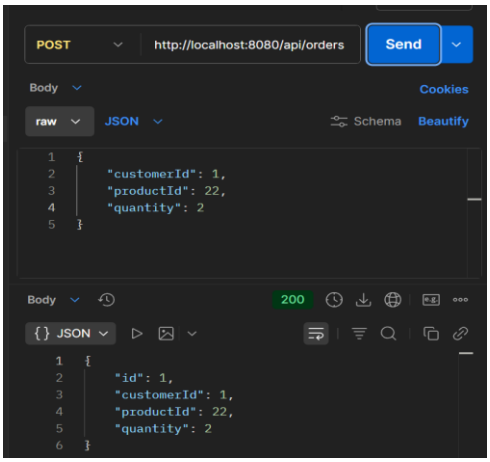
Méthode	URL	Description	HTTP	Module
GET	`/api/v2/products`	Liste tous les produits	**200 OK**	**Product**
POST	`/api/v2/products`	Créer un produit	**201 Created**	**Product**
PUT	`/api/v2/products/{id}`	Modifier un produit	**200 OK**	**Product**
POST	`/api/v2/customers`	Ajouter un client	**200 OK**	**Product**
POST	`/api/v2/orders`	Créer une commande	**201 Created**	**Order**


New R
Save
Share
GET
http://localhost:808
Send
Body
Cookies
raw
JSON
Schema
Beautify
Ctrl+Alt+P to Ask AI
Body
200
JSON
[
{
"id": 3,
"name": "Laptop",
"description": "Gaming PC",
"price": 1500.00,
"stock": null
},


My ...
New Request
Save
Share
POST
http://localhost:8080/api/produ...
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"name": "Ordinateur Portable",
"description": "Un PC puissant",
"price": 1200.50,
"stock": 5
}
Body
201
JSON
{
"id": 22,
"name": "Ordinateur Portable",
"description": "Un PC puissant",
"price": 1200.50,
"stock": 5


PUT
http://localhost:8080/api/produ...
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"name": "Ordinateur Portable",
"description": "Un PC puissant",
"price": 1200.50,
"stock": 6
}
Body
200
JSON
{
"id": 22,
"name": "Ordinateur Portable",
"description": "Un PC puissant",
"price": 1200.50,
"stock": 6
}


My ...
New Request
Save
Share
POST
http://localhost:8080/api/custo ...
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"name": "chaymae",
"email": "chaymaealaoui@gmail.com"
}
Body
200
JSON
{
"id": 1,
"name": "chaymae",
"email": "chaymaealaoui@gmail.com"
}


POST
http://localhost:8080/api/orders
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"customerId": 1,
"productId": 22,
"quantity": 2
}
Body
200
JSON
{
"id": 1,
"customerId": 1,
"productId": 22,
"quantity": 2
}

7. Comparaison des Architectures

Ce TP illustre la progression naturelle entre trois styles architecturaux :

Critère	Monolithe (TP 1)	Monolithe Modulaire (TP 2)	Microservices (futur)
Déploiement	1 artefact	**1 artefact**	N artefacts indépendants
Couplage	Fort	**Faible (interfaces)**	Très faible (réseau)
DTOs	Optionnels	**Obligatoires**	Obligatoires
Communication	Directe (classes)	**Via interfaces Java**	Via HTTP/gRPC/Kafka
Complexité ops.	Faible	**Faible**	Élevée
Scalabilité	Globale uniquement	**Globale uniquement**	Par service
Recommandation	Prototypes	**Applications métier**	Très large échelle

8. Conclusion

Ce TP a permis de maîtriser la **restructuration d'un monolithe classique** en une **architecture modulaire propre**. Les quatre techniques clés mises en œuvre — modularisation par domaine, DTOs, MapStruct et interfaces de services publics — forment un socle robuste qui prépare l'application à une éventuelle migration vers des microservices.

MapStruct a démontré sa supériorité sur les approches manuelles ou par réflexion, en offrant des performances natives et une détection des erreurs à la compilation. Le pattern **Interface + Implémentation** garantit que chaque module expose un contrat stable, indépendant de ses détails internes.

> **Prochaine étape :** La migration vers les microservices consistera principalement à extraire chaque module en un service Spring Boot indépendant, en remplaçant les appels d'interfaces Java par des appels HTTP (RestTemplate / WebClient) ou des messages asynchrones (Kafka / RabbitMQ). Le travail de modularisation effectué ici rend cette transition naturelle.

Rapport TP 2 — Module Architecture Logicielle & Spring Boot — 2026